

Приложение на циклични конструкции. Цикъл while

Работен лист — задачи от урока

Инструкции за работа

Моля, прочетете внимателно условията на задачите. Изпълнявайте програмните кодове в избраната от вас среда за програмиране на Python (напр. IDLE, PyCharm или Thonny) и записвайте получените резултати в предвидените за това места. Обръщайте специално внимание на синтаксиса и индентацията (отместването) на командите.

1. Въведение в цикъла while

Понякога еднократното изпълнение на определени команди не е достатъчно, за да се получи желаният резултат. Затова в езиците за програмиране съществуват специални конструкции, които позволяват многократното изпълнение на група от команди – т.нар. „цикли“. В Python една от основните възможности за реализиране на цикли е чрез командата `while`. Идеята ѝ е да осигури повторението на определен код, докато е изпълнено (вярно) дадено условие.

Синтаксис на командата:

```
while условие:
    команда 1
    команда 2
    .....
```

Важно за индентацията!

В Python отместването (индентацията) надясно след символа „:“ е задължително. То определя кои команди принадлежат към тялото на цикъла и ще бъдат изпълнявани многократно.

Задача 1: Изследване на програмен код и проследяване

Въведете следния код във вашата среда за програмиране и го стартирайте:

```
number = 0
while number < 5:
    print(number)
    number += 1
print('Сега стойността на брояча е 5')
```

Таблица за проследяване (Таблица 1): Анализирайте как се променят стойностите при всяко повторение. Попълнете липсващите данни в последните две стъпки.

Стъпка	Условие	Действие
1-ва	True	Отпечатва се 0. Броят number става 1.
2-ра	True	Отпечатва се 1. Броят number става 2.
3-та	True	Отпечатва се 2. Броят number става 3.

Стъпка	Условие	Действие
4-та	True	Отпечатва се 3. Броят number става 4.
5-а		
6-а		

Какъв е крайният изход в конзолата от целия код?

(Запишете тук резултата, който се изписва на екрана)

Задача 2: Модификация на цикъл

Напишете програма, която използва while, за да отпечата числата от 1 до 9 в конзолата.

Насока: Помислете каква трябва да бъде началната стойност на променливата и как да формулирате условието, така че цикълът да спре точно след числото 9.

Вашият код:

```
# Напишете решението на Задача 2 тук
```

Задача 3: Проверка на парола (Интерактивен цикъл)

Допълнете програма за въвеждане на парола, която да пита потребителя за вход и да повтаря процеса, докато не бъде въведена правилната парола.

Въпрос: Какво трябва да е условието на цикъла, за да продължи той да се изпълнява при всяко въвеждане на грешна парола?

(Опишете логиката на условието тук)

Задача 4: Брояч на грешни опити

Променете логиката на Задача 3, така че програмата да извежда съобщение "Твърде много грешки!", ако потребителят направи повече от 3 грешни опита.

Съвет: Дефинирайте променлива-брояч с начална стойност 0 преди цикъла. В края на цикъла увеличавайте този брояч с 1.

Запишете фрагмента от кода, който прави проверка дали броячът е станал > 3:

```
# Напишете само if проверката тук
```

6. Команди Break и Continue и Задача 5

В Python съществуват „властелини на цикъла“, които позволяват по-прецизен контрол:

- **break:** Прекъсва целия цикъл веднага и предава контрола на следващата команда след него.

- **continue:** Пропуска остатъка от текущата стъпка и се връща в началото на цикъла за нова проверка на условието.

Задача 5

Добавете сигурност към Задача 3. От съображения за сигурност след 5-ия неуспешен опит за въвеждане трябва да се изведе съобщение "Вашият акаунт ще бъде заключен!" и цикълът да се прекъсне незабавно.

Вашият код (използвайте if и break вътре в цикъла):

```
# Напишете решението на Задача 5 тук
```

7. Предизвикателство: „Опитайте се да решите“ (Turtle Graphics)

Използвайте модула turtle, за да начертаете фигура от завъртени квадрати. Попълнете липсващите части (отбелязани с __), като използвате дефинирания списък с цветове и геометрията на квадрата.

```
from turtle import *
from random import *

tina = Turtle()
tina.speed(0)
colors = ['yellow', 'orange', 'red', 'purple', 'blue', 'green']

# Функция за начертаване на квадрат
def square():
    for x in range(4):
        tina.fd(__) # Попълнете дължина на страната (напр. 100)
        tina.left(__) # Попълнете ъгъла за завой в квадрат

x = 0
while x < 360:
    tina.color(choice(__)) # Попълнете името на списъка с цветове (colors)
    square()
    x += __ # Попълнете със стъпката на въртене (напр. 10)
    tina.setheading(x)
```

Инструкция: Изпробвайте кода в Python. Наблюдавайте как се променя гъстотата на фигурата, ако промените числото, с което увеличавате x в края на цикъла.